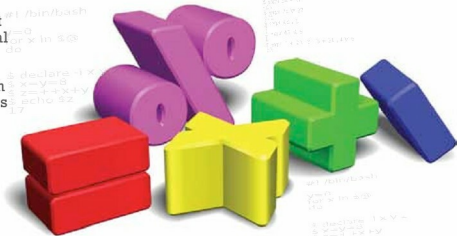


A Mathematical Journey Into The Shell

This article is the first part of a mathematical journey in the open source world and takes readers through the basic mathematics in a shell.



Mathematics is part of every moment of our lives, starting from measuring the time, to shopping, to advanced engineering calculations and computations. We have learned math throughout our childhood by counting. With the age of computers, we have been taught how to use them for mathematical purposes. So let's explore the mathematical capabilities of various open source software. Today, we start with the most preliminary one—the shell.

Shell command 'expr'

This supports basic operations: addition, subtraction, multiplication (*), quotient (/), remainder (%) with C-like precedence and associativity rules. Some usage examples follow:

```
$ expr 34 - 67
-33
$ expr 23 \* 27
621
$ expr 43 / 6
7
$ expr 43 % 6
1
$ expr 2 + 2 \* 3 - 5 + 21 / 4 \* 6
33
```

Note the spaces between `expr`, numbers, and the operators—`expr` is space-sensitive. The power of this simple command is its 'precision-less-ness'. Try out the following to really understand what I mean:

```
$ expr 1024 \* 1024 \* 1024 \* 1024 \* 1024 \* 1024 \* 1024 \* 1024 \*
```

```
1024 \* 1024 \* 1024
```

You still don't get an overflow, but the result is 126765 0600228229401496703205376, i.e., 2^{100} . Also, you may use variables. For example:

```
$ x=5; y=6; z='expr $x + $y'
$ echo $z
11
```

However, it has limitations; first, you do not have the exponentiation operator. Moreover, you can't change precedence by using brackets. And that's where you start looking at the other options.

Shell's arithmetic expansion

With this, you get all the mathematical C operators, plus exponentiation using `**`. You have bracketing, and variable usage even without the cryptic `$`. It is achieved by putting the complete expression to be evaluated, between `$((...))`, which is the arithmetic expander. Here's a set of examples:

```
$ echo $((2+3)*4-2**4/5%6))
17
$ x=y=8; echo $((3 << 4 | x | 2 << y))
568
```

There are some additional interesting ways to assign variables, as well, using `let` or `declare -i`: